→ Reinforcement learning projects with OpenAI gym

py -m venv .mynewenv | .\mynewenv\Scripts\Activate.ps1

(base) → (.mynewenv)

Python 3.10.11

↳ pip install gym==0.25.2 tensorflow==2.12.0 keras-rl2 numpy pygame

↳ pip list

Cmd

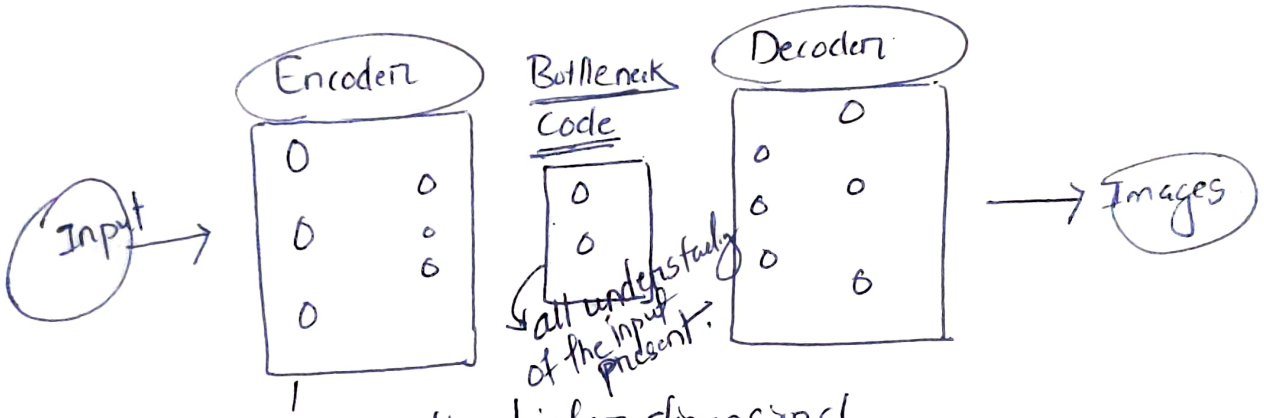
D:

cd D:\reinforcement_learning

code .

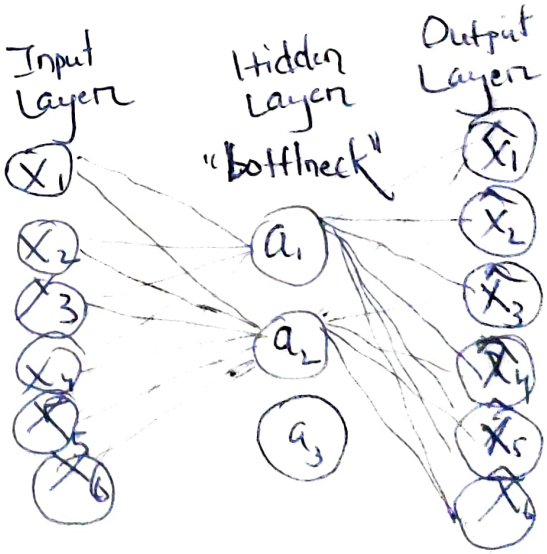
Ctrl + Shift + ~ — New Terminal

AUTO-ENCODERS



Encoder : making the higher dimensional data into lower dimensions.

Decoder : decoding lower dimensional data into higher dimensions.

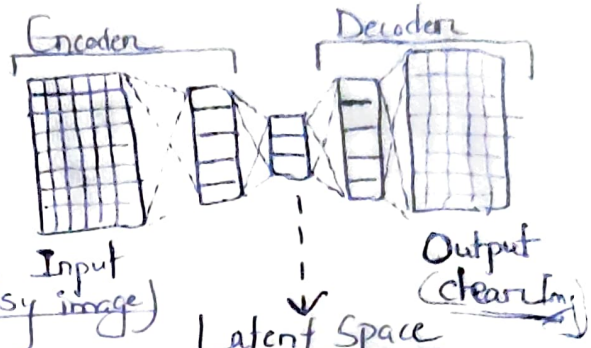


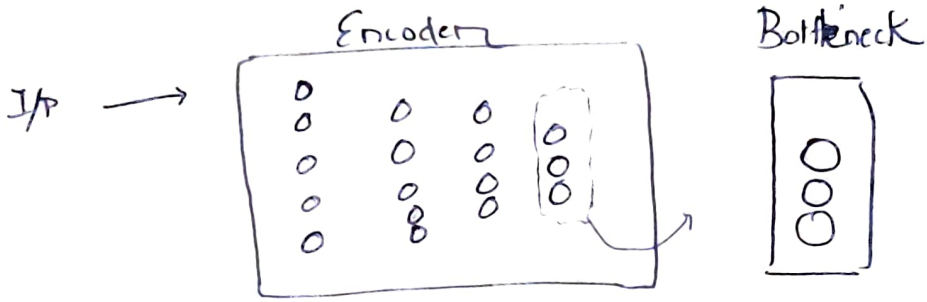
- UnSupervised learning
- Consists of Encoder and decoder
- The encoder transforms the input data into a reduced-dimensional representation,
- which is often referred to as "Latent Space" or "encoding".
- A decoder rebuilds the initial inputs from that representation.
- The general architecture of an auto-encoder includes encoder, decoder, and bottleneck layer.

Basic Structure and Components of Autoencoders

Encoder :-

- Input Layer take raw input data.
- The hidden Layers progressively reduce the dimensionality of the input, capturing import features and Patterns. These compose the encoder.
- The bottleneck Layer (latent Space) is the final hidden Layer, where the dimensionality is significantly reduced. This Layer represents the compressed encoding of the input data.





Bottleneck :-

↳ The bottleneck, also known as "code", is the most compressed representation of the Input.

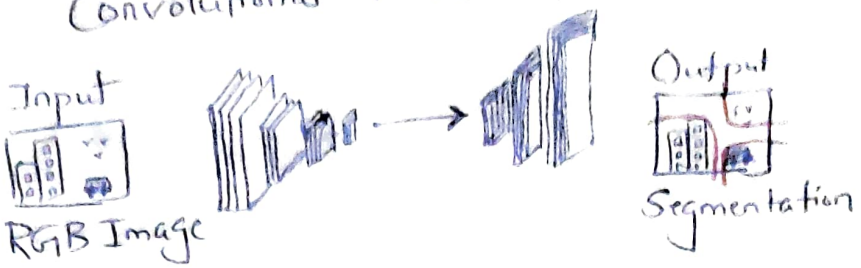
↳ it Serves as the Output Layer of Encoder & Input Layer for Decoder network

- A Primary Objective in designing & Training an Autoencoder is to Identify the minimum number of crucial features or dimensions necessary for reconstructing the Input data effectively.
- The Latent Space representation, which is the code, is generated from this bottleneck layer and is subsequently passed into the decoder.

Decoder :-

- The bottleneck layer takes the encoded representation and expands it back to the dimensionality of the Original Input.
- The Hidden Layer Progressively increases the dimensionality and aim to reconstruct the Original Input.
- The Output Layer Produces the reconstructed Output, which ideally should be as close as possible to the input data.

Convolutional - encoder - decoders

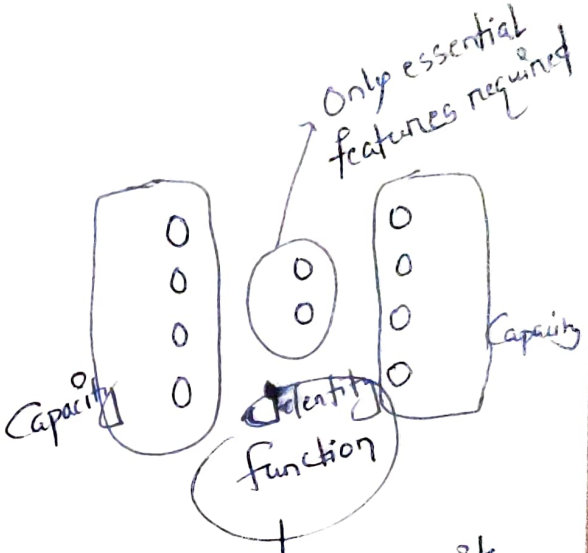


Types of Auto-Encoders :-

Based on Tasks

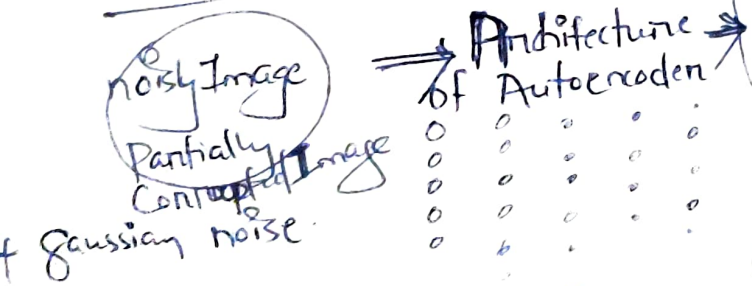
(1) Under-Complete Autoencoders :

- aim to learn a compressed representation of input data.
- perform dimensionality reduction by forcing the network to capture the most salient features.
- Used in feature learning, anomaly detection, and denoising tasks.



When bottleneck consists of all features (~~except~~ Instead of essential feature) the Auto Encoder is Overfit. decoder can easily generate input data clearly. (mapping output from input to memorizing traing data.

(2) Denoising-Autoencoder :



Reconstructed Loss

Compare with denoised image with the output.

→ Denoising Autoencoders are Learned/Trained such that it reconstruct the Original Image by removing/filtering noise.

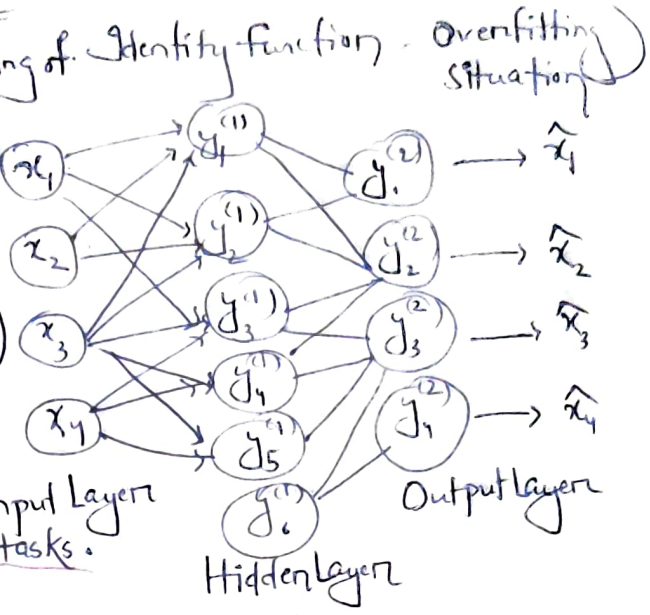
- Works on Partially Corrupted Input.
- Trains to recover the Original undistorted Image.
- It can extract important features and reduce the noise or the useless features.

③ Sparse Autoencoders

↳ Regularized Autoencoders

(To make sure no mapping of Identity function - Overfitting situation)

- Learns Sparse data representation
- Emphasizes few active hidden neurons.
- Uses Regularization (L1 penalty) (λ, Sparsity constraint) weight → 0
- Encourages feature Extraction.
- Common in dimensionality Reduction tasks.



Sparse features

④ Variational AutoEncoders : (VAE)

↳ generating new data point, by receiving just a little bit of noise as input.

↳ Generating model learning Compress representation of Training data as a probability distribution. It utilizes Learned representation to generate new sample data by Creating variations.

→ VAE Learns continious Latent model variables.

→ AE → (2,4) discrete Vector representation (encoded in)

① VAE Cencoded in μ Vector, σ Vector) → tells about Gaussian distribution

Interpolation - ability to Create new datapoint between existing one

enables Stochastic encoding

random Sampling

Interpolation & random Sampling expanding Capability of VAE comes from

(2) VAE is Generative in nature.

Discrete latent Space

(2,3)

Specific value of a encoded vector.

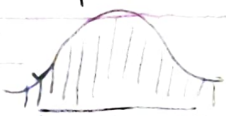
Continious latent Space (VAE)

(2.2, 3.3)

it output range of values as it output μ vector, σ vector

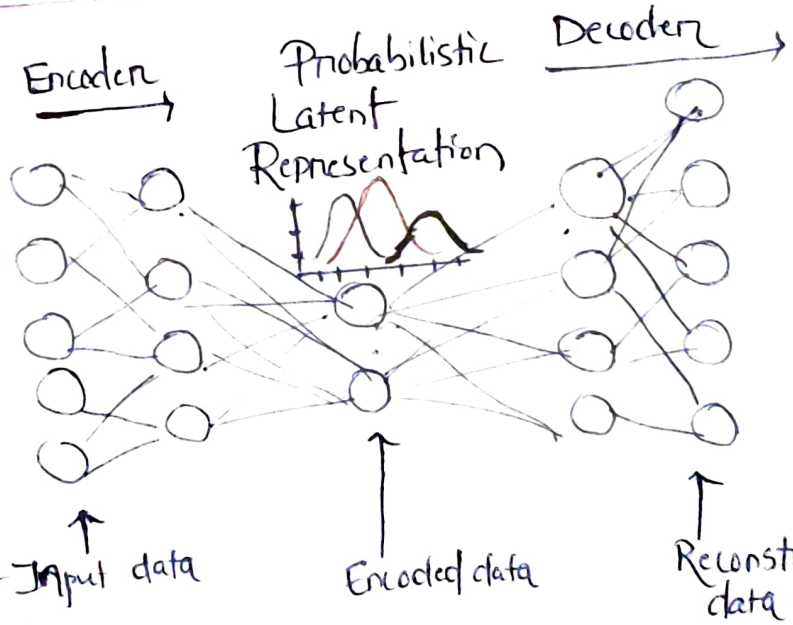
μ → σ →

these vector represents the Parameters of Probability distribution.



So, that machine do random sampling

- Variational Autoencoder (VAE) Learns latent representations.
- Balances reconstruction accuracy and Latent space continuity.
- Encoder maps input data to Latent distribution.
- Decoder generates data from sampled Latent variables.

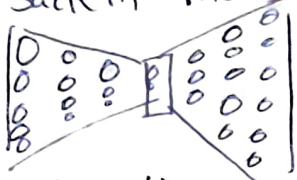


→ Tring to get Vectors (μ, σ Vectors) then Interpolation is done & random Sampling is done, then it is taken out as an Output. Then New data point has been generated.

→ Training Autoencoders : Loss Functions & Optimization Techniques.

Q) How to train Autoencoders?

- Code Size : # of nodes in the middle layer (Small size → more compression)
- # of Layers : defines the complexity of model.
- # of nodes per layer : # of neuron decreases with each subsequent layer of encoder & increases back in the decoder.



- Loss function / Reconstruction loss : MSE (for Continious data), BCE (Binary data)

if input value in the range [0,1] → cross-entrop.
otherwise MSE

→ The loss function which measures the reconstruction loss between output & Input is used to Optimize model's weights through Gradient descent during backpropagation.

Optimization Techique :

① Keep Small Hidden Layers : (force to pick only the representati features of the data thus encoding the data)

② Regularization : Loss term is added to the Cost function

③ Denoising : (add noise to input data & teach the Network how to remove noise)

④ Tuning the Activation Function : This method involves changing the activation functions of various nodes so that a majority of the nodes are dormant thus, effectively reducing the size of the hidden layers.

not active for some ti